web10: User-Owned Social Data with IAM-Grade Access Contracts at Zero Scan Penalty

Jacob Hoffman
web10 Inc.
jacob@web10.app

ABSTRACT

User-owned-data protocols such as Solid and ActivityPub defined the idea— your data, your rules—but stopped at the specification: container-level access control, no group/role model, and storage that cannot run a real social feed. They produced a protocol without a product.

We present **web10**, a next-generation user-owned-data protocol *and* its execution: a running social platform (feeds, groups, direct messages, discovery) built end to end on a modern columnar data-lakehouse backend. Access is governed by fine-grained, revocable, group/role-scoped contracts—cloud IAM primitives brought to the *front* of the user—enforced at the query layer so fine-grained authorization costs an index lookup, not a scan.

The application is the research. In a single query-engine call, web10 returns a full feed of **260 assets**, each access-checked against the reader's role grants: a large, tunable feed and fine-grained user-gated access coexisting in one query. That conjunction is what prior systems proposed but could not ship.

1 INTRODUCTION

The dominant social platforms solve engagement extremely well and ownership extremely poorly. A user's posts, messages, and relationships live in the platform's database under the platform's access model. The user can adjust a handful of visibility settings the platform chose to expose; they cannot redefine who a "friend" or a "member" is, cannot contract access per-service, and cannot take their data—let alone their audience— elsewhere. When the platform changes its rules or goes away, the user's data goes with it.

Cloud infrastructure solved the authorization half of this problem long ago. AWS IAM and its descendants give operators fine-grained, role-based, service-scoped, revocable access control with cryptographic token verification. But that model lives at the *infrastructure* boundary: the cloud provider owns the data plane, and the end user is invisible to it. Nobody has brought the IAM model to the *user* boundary of a social platform—where the user, not the platform, is the principal that holds the contracts.

Every serious attempt at user-owned social data has stopped at the same wall, and the gaps are specific. **Solid** gives users personal data stores (Pods) and a body of work on building social applications over them, but its access control is WebACL—permissions attached to *containers*, with no first-class group/role primitive and no way for a person to request, get approved, and receive a per-service permission set; and its per-Pod storage makes a cross-user feed query intractable, so those social applications remain hypothetical. **ActivityPub** and **ATProto** federate the activity but leave authorization to federation-trust rather than fine-grained, user-held contracts. **Cloud IAM** has exactly the permission language a social platform needs—principals, roles, service-scoped operations, revocable grants—but it lives at the infrastructure boundary, where the provider owns the data and the end user is invisible.

What none of them have is what web10 ships: a *next-generation* user-owned-data protocol *and its execution*—the protocol and the product as one thing, built end to end on a modern columnar data-lakehouse backend. Concretely: user-owned data, with IAM-grade group/role access enforced at the query layer of a single columnar store, in a running social platform. Ownership without sacrificing the feed; fine-grained access without a scan penalty; and a product, not a protocol proposal. That conjunction—and the architecture that makes it possible—is this paper.

web10 is a social data platform that makes that move. Every user's data lives under their own key in a single columnar store; every access decision is a check against two user-held contract layers; and the social application (feeds, groups, direct messages, discovery) is built entirely from these primitives with no app-specific surface in the node. This paper makes three contributions:

(1) **A two-layer access model for user-owned social data** (§2): an outer *app trust* layer (per-origin, per-service operation grants) and an inner *person trust* layer (group roles as per-service permission maps over three nested principal classes). Both must pass; either can deny.
(2) **A query-layer enforcement design** (§3) in which the role check never touches the large document table. Effective permissions are resolved against small metadata tables, producing a list of cleared group identifiers that the columnar scan consumes as an ordinary filter. Fine-grained control, zero scan-time penalty.
(3) **A running social platform** (§4) built entirely on these primitives— feeds, groups, direct messages, discovery—in which a single query-engine call returns a full feed of 260 assets, each access-checked against the reader's effective group/role grants. This is the commercial-grade execution of user-owned social data that prior protocol- first systems proposed but did not ship.

The remainder of this paper: §2 defines the access model; §3 shows how it is enforced over one columnar table; §4 presents the running social platform and its feed as the demonstration; §5 positions web10 against Solid, federated social protocols, and cloud IAM; §6 concludes.

2 THE ACCESS MODEL

2.1 Two trust layers

Every request in web10 names an *app* (the calling origin), a *principal* (the acting user), a *group* G, a *service* S (a data namespace such as `posts` or `comments`), and an *operation* O (e.g. `readAll`, `create`). Authorization asks two independent questions, and *both* must pass:

Layer	Question
App trust (outer wall)	Can this *app* perform O on S?
Person trust (inner wall)	Can this *principal*'s effective role in G grant O on S?

The app contract is *infrastructure trust*: which origins may touch which services, enforced by CORS in the browser and re-checked server-side on every call. The group role is *social trust*: which people may do what inside a group. The split matters because the two layers change at different rates and for different reasons—a user re-points an app's access without leaving any group, and leaves a group without touching any app's access.

2.2 Roles are per-service permission maps

A role is a named bundle of per-service permissions—a map from service to the operations it grants:

```
{ "name": "editor",
  "permissions": {
    "posts":    ["readAll", "create", "updateOwn"],
    "comments": ["readAll"]
  } }
```

The map *is* the scope. `'*'` wildcards all document services; a reserved key `'group'` carries *structural* operations on the group itself (`manageRoles`, `assignRoles`, `deleteGroup`, join/member operations), while *content* operations—including moderation such as `hideAll`—live under the service key, because they act on documents, not on the group's shape.

Two properties keep the model small. First, the role map is the *same shape* as the app-contract permission object: there is one permission language across both trust layers. Second, a person holds *exactly one role* per group; the per-service map makes a single role fully expressive over any (principal, service, operation) matrix, so there is no multi-role stacking to reason about. Roles are generic—a group defines whatever roles fit its purpose (`owner`, `moderator`, `page-curator`, `member`, or `dj`, `listener`)— and the platform never names them.

2.3 Principal classes and effective permissions

Access is granted to *principals*, of which there are three nested classes, broadest to narrowest:

Class	Who
`anyone`	every request, signed in or not
`authenticated`	any valid token (any user, member or not)
`member`	holds a member row in this group

The classes are *reserved keys in the member table*—no separate visibility table exists. A group grants a class a role by storing a member row for the reserved key:

```
group_members:
  (G, 'anyone',        'reader')    -- the group is
      public
  (G, 'authenticated', 'reader')    -- signed-in users
      can read
  (G, 'bob',           'moderator') -- bob moderates
  (G, 'alice',         'owner')     -- alice owns
```

A principal's *effective* permissions in a group are the union of the permission maps of every class they belong to:

$$\begin{aligned}\text{effective}(P, G) \;=\; &\text{grant}(G,\ \texttt{anyone}) \\ \cup\ &(P \in \texttt{authenticated}\ ?\ \text{grant}(G,\ \texttt{authenticated}) : \emptyset) \\ \cup\ &(P \in \texttt{member}(G)\ ?\ \text{grant}(G,\ \text{role}(P)) : \emptyset)\end{aligned}$$

The nesting buys a monotonicity invariant for free: a member always sees at least what a signed-in stranger sees, who always sees at least what a signed-out visitor sees. The model cannot express the inversion.

2.4 Publicness is a grant, not a flag

There is no `visibility` column. "Make my group public" is the act of granting `readAll` to `anyone`; "signed-in only" grants it to `authenticated`; "private" grants it to member roles only. Join policy is orthogonal: it controls how a human *becomes* a member (open, request, invite-only) and says nothing about what a non-member bystander can see. A group can be invite-only to join and still publicly readable—the discovery board is exactly that.

This is the sense in which web10 brings IAM to the front: the operations a cloud operator would express as policies—*principal class, service, operation, grant*—are the operations an *end user* expresses about their own data, with the same revocability (a grant is a row; removing it is a tombstone that takes effect on the next read).

3 ENFORCEMENT AT THE QUERY LAYER

3.1 One columnar table

All user content lives in a single ClickHouse table, keyed by `(author_key, doc_id)`:

```
CREATE TABLE documents (
    doc_id          String,
    author_key      String,
    collection_name String,   -- the service: 'posts', '
        comments', ...
    body            String,   -- JSON
    tags            Array(String),
    created_at      DateTime64(3),
    updated_at      DateTime64(3),
    deleted         UInt8 DEFAULT 0
) ENGINE = ReplacingMergeTree(updated_at)
ORDER BY (author_key, doc_id);
```

Updates are inserts with a higher `updated_at` (the engine keeps the latest version); deletes are tombstone rows filtered at read time. Documents attach to groups through a small mapping table, and groups carry their contracts and members in two more small tables. Content is one wide columnar table; *access is a handful of narrow metadata tables.*

3.2 The role check never touches the document table

The read path is the heart of the design:

(1) **Resolve.** Look up the reader's rows in `group_members` (including the reserved principal-class rows) and the role maps in

`group_contracts`. These are small, hot, and indexed by their primary keys.

(2) **Compute.** For the requested service, compute the set of group identifiers whose effective role grants `readAll` to the reader.

(3) **Scan.** Run the columnar `documents` scan exactly as it always would, with the cleared-group list as an ordinary filter, unioned with the reader's own `author_key`.

The authorization decision is therefore an index lookup into small metadata tables, not a per-document check inside the scan. The scan stays a scan. This is the "zero scan-time penalty" property: fine-grained, per-group, per-service control costs the same as the coarsest possible filter, because it *is* a filter list computed once per query.

3.3 The same invariant on every path

The enforcement rule is uniform: *no query returns documents for an* `author_key` *the token does not own, unless group membership grants access* (invariant I3). Every read path—group feed, read-by-id, directory search—applies the identical principal-class gate, and the write side is gated the same way, so a bystander cannot attach content to a group they cannot write. A conformance suite of permission-matrix cases (including replayed-exploit denials) enforces the invariants mechanically; the full suite results appear in the extended version.

4 A RUNNING SOCIAL PLATFORM

The strongest evidence for the model is not a benchmark but a product: web10 runs a complete social application—feeds, groups, direct messages, and discovery—entirely on the primitives of §2 and §3, with no app-specific surface in the node (D60). This is the commercial-grade execution of user-owned social data that protocol- first systems such as Solid proposed but did not ship. We highlight the feed, because it is the workload that most stresses the design: a real social feed must be large, tunable, and access-controlled simultaneously.

4.1 The feed as a query

A single call to the query engine returns a full feed of **260 assets** (posts, media, and comments) for the reader. Three properties hold at once in that one call:

(1) **It is a real feed, not a demo.** 260 assets is a commercial- grade feed payload, not a three-item sample. The feed is a SQL query over the single columnar `documents` table, unioned across the reader's own key-space and their cleared groups.

(2) **It is fully user-gated.** Every one of the 260 assets passed the access model on its way out: the query engine applied invariant I3 (no document for an `author_key` the token does not own) plus the reader's effective group/role grants (§2). The result is not 260 arbitrary documents; it is exactly the set the reader's role clears. Fine-grained social access and a large feed coexist in one query—the conjunction that is this paper's core claim.

(3) **It is tunable.** The feed's shape—what it contains, how it is ranked and filtered—is a query the *application* writes, not logic hardcoded in the node. A different query is a different feed. Because the substrate is a single columnar store, ranking, filtering, engagement counts, and discovery are all ordinary queries the app can compose, which is what makes a Facebook-shaped feed achievable on user-owned data.

This is the "because of the columnar backend" point made concrete: Solid's per-Pod architecture makes a cross-user feed query intractable (each Pod is a separate server), so its social applications remain hypothetical. web10's single table makes the feed a first-class, tunable query while the access model stays a filter list with zero scan penalty.

4.2 Durability and extensibility

Because the node exposes only universal primitives and the access model is a per-service role map, the social app is not a one-off demo—it is the hardest instance of a general pattern. The same substrate supports other app shapes (a messaging app, a music app, a shop) by defining new services and roles over the same tables, with no node changes. The social platform demonstrates that the model carries a demanding, real workload; extensibility to other apps follows from the same genericity.

4.3 Scale and latency (extended version)

The deployed system's scale and access-decision latency are reported in the extended version: [**N**] registered users, [**N**] documents across [**N**] services and [**N**] groups, [**N**] access decisions per day; the resolve+compute phase (metadata lookup + effective-role computation) completes in [**X**] ms at p50 / [**Y**] ms at p99, a fraction [**Z**]% of total read latency. The extended version also adds the cost of the per-user key-space model versus a shared table with row-level security, end-to-end flow latencies against a conventional backend, and the full permission-matrix conformance results.

5 RELATED WORK

Decentralized social data. Solid [1] is web10's closest philosophical ancestor and the system most worth contrasting against. Solid gives users personal data stores (Pods) and has produced a body of work on building social applications on top of them; access is governed by WebACL—permissions attached to *resources and containers*—plus scoped app tokens. ActivityPub [2] and ATProto [3] federate social activity with user-held datastores. All three establish that a user can *own* their data.

The gap is authorization granularity. Solid's WebACL answers "who may read or write this container?"—a coarse, resource-level question. It has no first-class *group* primitive with *roles*, and no *request* → *approve* → *assign-role* lifecycle: there is no notion of a person asking to join a community and an owner granting them a specific, per-service permission set. web10's contribution is exactly this missing layer. Access in web10 is *person* → *role* → *per-service operations*, granted *inside* a group through an explicit join-request and approval flow, where a role is a map from data service to the operations it grants (§2). The contrast is not storage (one columnar table versus per-Pod LDPC containers) though that differs too; it is that web10 makes fine-grained, role-scoped, per-service *group*

authorization—the IAM shape—a first-class, query-enforced primitive with a measured scan cost, and does so on top of a *running* social application rather than a hypothetical one.

Cloud IAM. AWS IAM [4] and the RBAC/ABAC literature [5] define the permission language web10 reuses at the user boundary: principals, roles, service-scoped operations, revocable grants. The move in this paper is directional—the same policy shape, held by the end user about their own data, enforced by the data layer rather than a control plane.

Multi-tenant isolation. The database literature on tenant isolation—schema-per-tenant, database-per-tenant, and row-level security [6]—is the closest systems ancestor. web10's key-space per user is tenant-per-end-user taken to its limit: the tenant is the *end user*, and the isolation is expressed as the query's `author_key` filter plus group-grant list rather than a physical boundary. The cost tradeoff of that choice is measured in the extended version.

6 CONCLUSION

web10 brings the cloud IAM permission model—principal classes, roles, service-scoped operations, revocable grants—to the user boundary of a social data platform, and enforces it at the query layer of a single columnar store so that fine-grained authorization costs an index lookup, not a scan. We demonstrate the model in a running social platform whose feed is a tunable query returning 260 fully access-checked assets in a single call—the commercial-grade execution of user-owned social data that protocol-first systems proposed but did not ship. The extended version adds scale and latency figures and the cost comparison against row-level-security baselines.

REFERENCES

[1] T. Berners-Lee. Solid: A distributed social web protocol. *Solid Project / W3C*, 2014–present. https://solidproject.org.

[2] M. Sporny, D. Wilson, R. Smith, and D. Sadok. ActivityPub: A protocol for distributed federated publishing, archiving, and sharing. *IETF Internet-Draft*, 2023.

[3] D. Holman and M. Roth. AT Protocol: A personal data protocol for social applications. *Bluesky Technologies*, 2022–present. https://github.com/bluesky-social/atproto.

[4] Amazon Web Services. AWS Identity and Access Management (IAM) user guide. *Amazon Web Services*, 2025. https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html.

[5] R. Sandhu, E. Coyne, H. Feinstein, and C. Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996.

[6] A. Kemper and S. Lott. Row-level security in PostgreSQL. *PostgreSQL Documentation*, 2024. https://www.postgresql.org/docs/current/ddl-rowsecurity.html.